

ZAPIER PIPELINES, ENGINEERED

Zapier: Automations that finally hold state.

When No-Code agility turns into a
financial and security bottleneck.



+117k

downloads

**<50
MB**

RAM, total

SQLite

WAL state

THE CONTEXT

Where it hurts today

Zaps break silently

No real retry policy

Data held hostage by a SaaS

THE SHIFT

What you gain



Code-first

Pipelines are Python — versionable, testable.



Resume, don't restart

Every step checkpoints to the WAL.

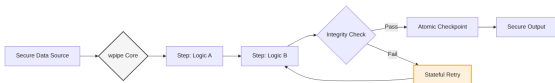


Featherweight

<50 MB. No broker, no daemon.

THE ARCHITECTURE

One pipeline, fully instrumented



State, retries and audit — inside a single run.

SIDE BY SIDE

wpipe vs. the status quo

Dimension	Zapier (SaaS)	wpipe (Software Engineering)
Data Sovereignty	Third-party dependency	Total (In-house / Local-first)
Operating Cost	Volume-based (Variable)	Predictable (Own infrastructure)
Robustness	Proprietary black box	Transparent (Real stack traces)
Resilience	Generic retries	SQLite Checkpoints (Persistent state)

SEE IT IN ACTION

Six lines. That's it.

```
from wpipe import Pipeline, step

@step(name="sync",
retry_count=3, retry_delay=5)
def sync(data):
    return {"done":
push_crm(data["lead"])}

pipe =
Pipeline(pipeline_name="zap2wpipe")
```

CHECKPOINTS · RETRIES · TIMEOUT ·
PARALLEL · TRACKER

Try wpipe in your next pipeline ✓

Install the library, wrap your step
— checkpoints and docs appear
by themselves.

```
pip install wpipe
```

wpipe

Python-native orchestration